

Cahier des Charges du projet

Réservation de Salle

Client : TechSpace Solutions

Version : 1.0

Date : 13 Février 2026

1. SYNOPSIS

1.1 Contexte

TechSpace Solutions est un espace de coworking accueillant environ 50 collaborateurs répartis en plusieurs équipes. L'entreprise dispose d'une unique salle de réunion (capacité 12 personnes) partagée.

1.2 Problème

La gestion actuelle des réservations est artisanale (post-its, emails, tableau blanc). Cela entraîne des conflits récurrents (doubles réservations) et une sous-utilisation de la salle faute de visibilité sur les créneaux libres.

1.3 Besoin

Le client souhaite une application web (Intranet) simple et efficace permettant aux collaborateurs de vérifier la disponibilité de la salle en temps réel et de la réserver de manière autonome, sans intervention d'un administrateur.

1.4 Contraintes Générales

- **Cible :** 50 collaborateurs sur ordinateur (Desktop).
 - **Délais :** Livraison d'une V1 fonctionnelle sous 2 semaines.
 - **Périmètre V1 :** Une seule salle, pas de gestion admin, pas d'envoi d'emails.
-

2. CAHIER DES CHARGES CLIENT - FRONT-END

2.1 Cahier des Charges Fonctionnel

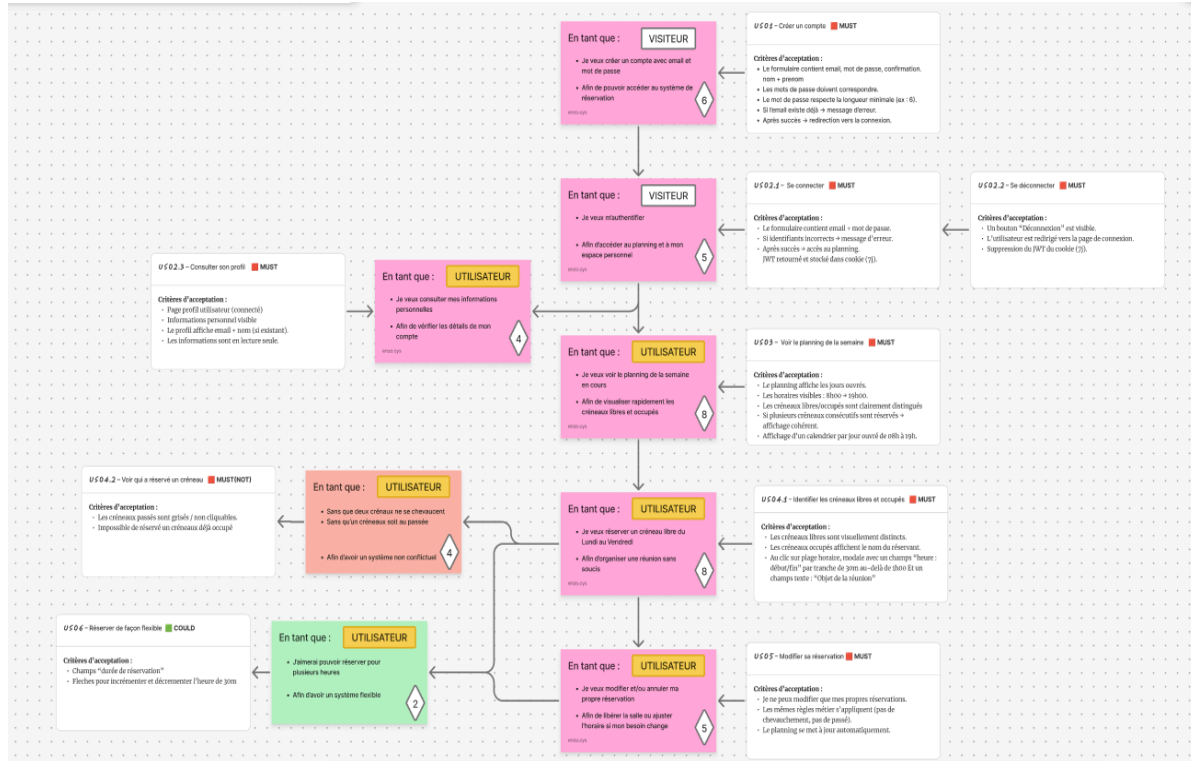
L'interface utilisateur doit être épurée, intuitive et réactive. Elle s'articule autour d'un planning visuel permettant d'identifier immédiatement les disponibilités.

2.1.1 User Stories (Priorisation MoSCoW)

Les priorités sont définies strictement selon le Brief Client.

ID	En tant que...	Je veux...	Afin de...	Priorité
US-01	Visiteur	Créer un compte (Nom, Prénom, Email, MDP)	Accéder à l'application.	MUST
US-02	Visiteur	Me connecter / Me déconnecter	M'identifier de manière sécurisée.	MUST
US-03	Utilisateur	Consulter mon profil	Vérifier mes informations personnelles.	MUST
US-04	Utilisateur	Voir le planning de la semaine (Lun-Ven)	Identifier les créneaux libres et occupés.	MUST
US-05	Utilisateur	Voir les détails d'une réservation	Savoir qui occupe la salle et le sujet de la réunion.	MUST
US-06	Utilisateur	Réserver un créneau libre	Bloquer la salle pour ma réunion.	MUST
US-07	Utilisateur	Modifier ma réservation	Changer l'horaire ou l'objet (si je suis l'organisateur).	MUST
US-08	Utilisateur	Annuler ma réservation	Libérer la salle si ma réunion est annulée.	MUST
US-09	Utilisateur	Supprimer mon compte	Retirer mes données personnelles (RGPD).	SHOULD

Note sur US-05 : Le brief précise explicitement "Voir qui a réservé chaque créneau".



2.1.2 Règles de Gestion (Contraintes Métier)

Le Front-end doit appliquer ces règles lors de la saisie (validation formulaires) :

1. **Jours Ouvrés** : Réservations possibles uniquement du **Lundi au Vendredi**.
2. **Plage Horaire** : Créneaux autorisés uniquement de **08h00 à 19h00**.
3. **Durée** : Minimum **1 heure** par réservation.
4. **Temporalité** : Impossible de réserver une date ou une heure passée.
5. **Chevauchement** : Un créneau ne peut pas être réservé s'il empiète sur une réservation existante.
6. **Permissions** : Seul le créateur d'une réservation peut la modifier ou l'annuler.

2.2 Cahier des Charges Non-Fonctionnel (Technique Front)

2.2.1 Stack Technique & Déploiement

- **Framework** : React + Vite (selon Starter Kit).
- **Langage** : JavaScript (ES Modules).
- **CSS** : Tailwind CSS (Optionnel selon Starter Kit, recommandé pour rapidité).

- **Build** : `npm run build`.

2.2.2 Routes & Navigation

- `/` : Page d'accueil / Landing page.
- `/login` : Formulaire de connexion.
- `/register` : Formulaire d'inscription.
- `/planning` (ou `/dashboard`) : Vue calendrier (Route Protégée).
- `/profile` : Gestion de compte (Route Protégée).
- **Protection** : Redirection automatique vers `/login` si accès non autorisé via le composant `PrivateRoute`.

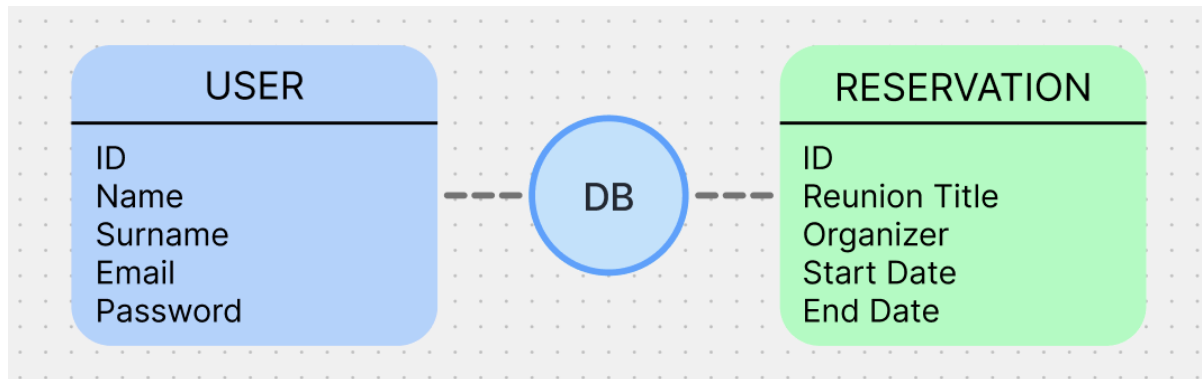
2.2.3 Sécurité Client

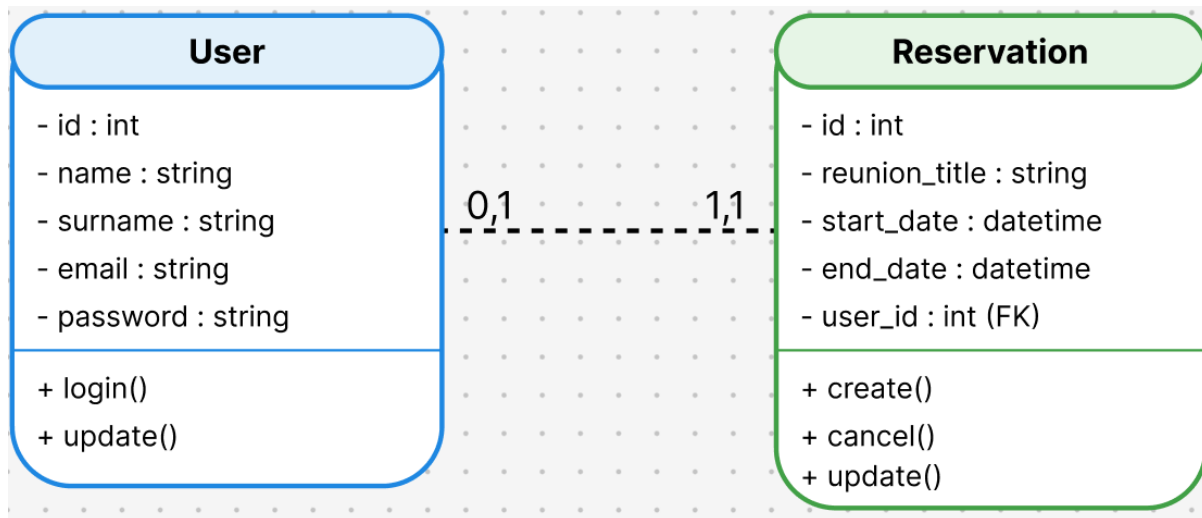
- Stockage du Token JWT dans le `localStorage` (implémentation Starter Kit).
- Expiration automatique de la session après 1 heure (validité token).

3. CAHIER DES CHARGES CLIENT - BACK-END

3.1 Cahier des Charges Fonctionnel

Le Backend doit fournir une API REST robuste capable de gérer la logique métier et l'intégrité des données pour 50 utilisateurs.





3.1.1 Besoin de Performance

- Support de ~10 utilisateurs simultanés sans latence.
- Réponse API rapide (< 200ms) pour l'affichage du planning.

3.2 Cahier des Charges Non-Fonctionnel (Technique Back)

3.2.1 Stack Technique

- **Serveur** : Node.js (v18+).
- **Framework** : Express.js.
- **Base de Données** : MySQL (Driver [mysql2](#)).
- **Architecture** : MVC (Model-View-Controller).

3.2.2 Endpoints API (Contrat d'Interface)

Verbe	Endpoint	Description	Auth	Codes Retour
POST	/api/auth/register	Inscription	Non	201, 400, 409
POST	/api/auth/login	Connexion	Non	200, 401
GET	/api/auth/me	Profil utilisateur	Oui	200, 401
DELETE	/api/auth/user	Suppression compte	Oui	200, 401
GET	/api/reservations	Liste des réservations	Oui	200, 401

Verbe	Endpoint	Description	Auth	Codes Retour
POST	/api/reservations	Créer réservation	Oui	201, 400, 409
PUT	/api/reservations/:id	Modifier réservation	Oui	200, 403, 404
DELETE	/api/reservations/:id	Annuler réservation	Oui	200, 403, 404

3.2.3 Modèle de Données (MCD Simplifié)

1. **Users** : `id`, `email`, `password`, `firstname`, `lastname`.
2. **Rooms** : `id`, `name`, `capacity` (Anticipation V2, unique salle "TechSpace" ID=1 pour la V1).
3. **Reservations** : `id`, `user\_id`, `room\_id`, `start\_time`, `end\_time`, `title`.

3.2.4 Contraintes de Sécurité Back-end

- **Authentication** : Middleware vérifiant le header `Authorization: Bearer <token>` sur toutes les routes protégées.
- **Validation** : Le Back-end doit revérifier TOUTES les règles métier (8h-19h, chevauchements, week-end) avant d'insérer en base de données. Ne jamais faire confiance au Front-end.
- **Injections SQL** : Utilisation stricte de requêtes préparées (?) via `mysql2`.
- **Mots de passe** : Hashage obligatoire via `bcrypt`.

3.2.5 RGPD & Confidentialité

- **Données collectées** : Minimisation (Nom, Prénom, Email uniquement).
- **Droit à l'oubli** : Lors de la suppression d'un compte (US-09), les réservations associées ne doivent pas être supprimées (pour éviter les trous dans le planning) mais **anonymisées** (ex: `user\_id` devient NULL).

4. ANNEXES

- **Maquettes** : À réaliser sur Figma (Desktop uniquement).
- **Script SQL** : Fourni dans le fichier `schema.sql`.
- **Collection Postman** : Fichier d'export JSON pour tester l'API.